

RASTRO

Transparencia y Datos Abiertos Colombia — Hackathon Croma

TECNOLOGÍAS USADAS

Stack Tecnológico

Herramientas, librerías y servicios que componen RASTRO, y por qué se eligieron.

Versión 1.0 — Agosto de 2026

Proyecto: RASTRO — motor de transparencia en contratación pública colombiana

Backend

Tecnología	Uso en RASTRO	Por qué
Node.js + Express	API REST pura (`server.js`), sin renderizado de HTML.	Simplicidad y control total sobre el ciclo de vida de cada request; separación estricta backend/frontend.
ES Modules nativos	Todo el backend usa `import`/`export` sin transpilador.	Node.js moderno soporta ESM de forma nativa — un paso menos de build para el hackathon.
@modelcontextprotocol/sdk	Cliente MCP que conecta el backend directo a Croma, sin pasar por un LLM en cada consulta.	Croma expone sus fuentes oficiales como herramientas MCP; el SDK es el cliente oficial.
jsonwebtoken + bcryptjs	Autenticación de cuentas de empresa (JWT) y hash de contraseñas.	Estándar de la industria, sin dependencias de infraestructura adicional (no requiere un servicio de sesiones aparte).
Persistencia en JSON (`jsonStore.js`)	Colecciones de empresas, outbox de correo/WhatsApp y oportunidades ya notificadas.	Suficiente para el volumen de un hackathon/demo; el patrón de acceso ya está aislado para migrar a Postgres si el volumen lo justifica.

Frontend

Tecnología	Uso en RASTRO	Por qué
Next.js 16 (App Router, Turbopack)	Único frontend activo (`web/`), Server Components por defecto.	Renderizado server-side para el contenido público (SEO, carga inicial rápida) con islas de cliente solo donde hay interactividad.
React 19	Librería de UI base de Next.js.	Versión estable más reciente al momento del desarrollo.
TypeScript	Todo el código de `web/` está tipado.	Los contratos de datos entre backend y frontend (Expediente, ContractTracking, etc.) se comparten como tipos explícitos, reduciendo errores de integración.
Tailwind CSS v4	Sistema de diseño basado en tokens (colores, tipografía, espaciado).	Permite mantener la identidad de marca (verde esmeralda / dorado) consistente en cientos de componentes sin duplicar valores.
Zustand	Estado de scroll/UI compartido entre componentes de animación.	Alternativa liviana a Redux para el estado de cliente puntual que necesita el sistema de animación.
@react-spring/web + Lenis	Motor de animación basado en springs y scroll suave.	Reglas de diseño del proyecto exigen animación basada en física (spring), no keyframes CSS, para una sensación más natural.

Tecnología	Uso en RASTRO	Por qué
Zod	Validación de entrada en cada route handler de <code>`app/api/**`</code> .	Valida y tipa la entrada del lado del servidor antes de reenviarla al backend Express.

Datos e integraciones externas

Tecnología	Uso en RASTRO	Por qué
Croma (MCP)	Fuente unificada de datos oficiales: SECOP, RUES, Rama Judicial, Procuraduría, Contraloría, Contaduría, ADRES, SIMIT, Policía Nacional.	Elimina la necesidad de integrar cada sistema gubernamental por separado.
Anthropic Claude API	Genera el resumen ejecutivo narrativo del expediente y del estudio de seguridad.	Modelo de lenguaje capaz de narrar evidencia ya clasificada sin inventar hallazgos ni emitir veredictos, siguiendo instrucciones estrictas del sistema.

Patrones de arquitectura

Tecnología	Uso en RASTRO	Por qué
Puertos y adaptadores (hexagonal)	<code>`GovDataSource`</code> , <code>`EmailPort`</code> , <code>`WhatsAppPort`</code> como interfaces; adaptadores reales y simulados intercambiables.	Permite degradar automáticamente a datos simulados si Croma falla, y conectar un proveedor real de correo/WhatsApp sin tocar la lógica de negocio.
Composición central de dependencias	<code>`dataSource.js`</code> como único punto donde se decide y envuelve la fuente de datos activa.	Caché y resiliencia se aplican una sola vez, de forma uniforme, a los 13+ métodos del puerto de datos.

Herramientas de desarrollo

Tecnología	Uso en RASTRO	Por qué
ESLint	Linting del código de <code>`web/`</code> .	Consistencia de estilo y detección temprana de errores.
Git / GitHub	Control de versiones del proyecto.	Estándar de la industria.
npm	Gestor de paquetes para ambos proyectos (<code>`/`</code> y <code>`web`</code>).	Cada proyecto mantiene su propio <code>`package.json`</code> y ciclo de vida independiente.